

Comprehensive Internship Project Report: Car Price Prediction with Machine Learning

Author / Intern: Data Science Intern

Internship Program: Oasis Infobyte Data Science Internship

Task Track: Task 3 Car Price Prediction with Machine Learning

Domain: Machine Learning Regression, Feature Engineering & Predictive Analytics

Associated Notebook: Car_Price_Prediction.ipynb

Date: September 2026

Executive Summary

For my third internship task, I built an end-to-end supervised machine learning regression pipeline designed to predict the resale market price of used cars based on vehicle age, current showroom price, mileage driven, fuel type, transmission type, seller channel, and brand manufacturer.

Predictive vehicle valuation is a core application in automotive e-commerce platforms (such as CarDekho, Cars24, or AutoTrader) that provides algorithmic pricing estimates for car sellers and buyers.

My technical workflow comprised:

1. Dataset Ingestion & Quality Audit
2. Duplicate Record Identification & Sanitization
3. Domain Feature Engineering (`Car\_Age`, `Brand`, `Price\_Depreciation`)
4. Visual Exploratory Data Analysis (Target KDE, Regplots & Category Boxplots)
5. One-Hot Encoding & Feature Correlation Analysis
6. Reproducible Scikit-Learn `ColumnTransformer` Pipeline Setup
7. Training 5 Regression Algorithms (Linear, Ridge, Decision Tree, Random Forest, Gradient Boosting)
8. Multi-Metric Performance Evaluation (MAE, RMSE, R^2 , 5-Fold Cross-Validation)
9. Best Model Feature Importance & Actual vs. Predicted Visual Validation
10. Production Model Serialization & Live Real-Time Inference Testing

The resulting **Random Forest Regressor** achieved an outstanding R^2 score of **0.9634** on the independent test set, with a low Mean Absolute Error (MAE) of **0.621 Lakhs INR** (~₹62,100), explaining over **96.3%** of the variance in used car selling prices.

Problem Statement & Business Context

1.1 The Pricing Problem

Determining the fair market value of a used automobile is complex due to competing depreciation factors:

- **Age Depreciation:** Cars lose value rapidly in their initial years.
- **Showroom Baseline:** Original vehicle tier (`Present_Price`) sets the ceiling for resale value.
- **Usage Wear:** Higher odometer readings (`Kms_Driven`) reduce mechanical value.
- **Engine & Transmission Dynamics:** Diesel engines and Automatic transmissions command distinct resale premiums in India.

1.2 Dataset Attributes

I worked with the CarDekho Used Vehicle Dataset (`car data.csv`) containing 301 records across 9 columns:

Attribute Name	Data Type	Description
Car_Name	Categorical	Manufacturer model name (e.g., <code>`ritz`</code> , <code>`sx4`</code> , <code>`ciaz`</code> , <code>`fortuner`</code>).
Year	Numerical	Year of original purchase (e.g., 2014, 2017).
Selling_Price	Numerical (Target)	Resale price in Lakhs INR (1 Lakh = ₹100,000).
Present_Price	Numerical	Current ex-showroom price in Lakhs INR.
Kms_Driven	Numerical	Total distance driven in kilometers.
Fuel_Type	Categorical	Fuel engine type: <code>`Petrol`</code> , <code>`Diesel`</code> , or <code>`CNG`</code> .
Seller_Type	Categorical	Seller category: <code>`Dealer`</code> or <code>`Individual`</code> .

Transmission	Categorical	Transmission mode: `Manual` or `Automatic`.
Owner	Numerical	Number of previous owners (0, 1, 3).

2. Data Cleaning & Sanitization (My Approach)

I performed a systematic data hygiene check:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
import joblib

warnings.filterwarnings('ignore')

# 1. Load dataset
df = pd.read_csv('car_data.csv')

# 2. Missing Value Audit
print("Null Values Count:", df.isnull().sum().sum()) # Output: 0 nulls

# 3. Duplicate Records Audit
duplicate_rows = df.duplicated().sum()
print("Duplicate Rows Found:", duplicate_rows) # Output: 2 duplicates

if duplicate_rows > 0:
    df = df.drop_duplicates().reset_index(drop=True)
    print("Cleaned Dataset Shape:", df.shape) # Output: (299, 9)

# 4. Categorical Case Normalization
for col in ['Fuel_Type', 'Seller_Type', 'Transmission']:
    df[col] = df[col].astype(str).str.strip().str.capitalize()
```

3. Feature Engineering

To help regression algorithms capture domain relationships, I engineered three new features:

11. **Car_Age**: Computed as `Current\_Year (2026) - Year`.

```
```python
df['Car_Age'] = 2026 - df['Year']
```

...

12. **Brand:** Extracted the parent brand manufacturer from `Car\_Name` (e.g., `maruti`, `hyundai`, `toyota`, `honda`, `ford`).

```
```python
```

```
df['Brand'] = df['Car_Name'].apply(lambda x: str(x).split()[0].lower())
```

...

13. **Price_Depreciation:** Calculated total value loss from original showroom price.

```
```python
```

```
df['Price_Depreciation'] = df['Present_Price'] - df['Selling_Price']
```

...

## 4. Exploratory Data Analysis & Visualizations

I generated four primary visual charts to uncover key price drivers:

### 1. Target Variable Distribution (`Selling\_Price`)

The histogram and KDE plot revealed a right-skewed distribution:

- **Mean Selling Price: 4.63 Lakhs INR**
- **Median Selling Price: 3.60 Lakhs INR**
- **Min / Max:** Range from **0.10 Lakhs** (older two-wheelers/economy cars) up to **35.00 Lakhs** (luxury SUVs like Toyota Fortuner).

### 2. Selling Price vs. Fuel Type (Boxplot)

- **Diesel:** Highest mean selling price (**10.27 Lakhs INR**) due to higher initial purchase price, torque, and commercial demand.
- **Petrol:** Mean selling price of **3.10 Lakhs INR**.
- **CNG:** Mean selling price of **3.10 Lakhs INR**.

### 3. Selling Price vs. Car Age (Scatter Plot with Regression Trendline)

Plotted using `sns.regplot`. The chart demonstrated a strong inverse relationship ( $r = -0.236$ ): vehicle prices drop sharply in the first 3 to 5 years, after which depreciation plateaus.

### 4. Selling Price vs. Transmission Type (Boxplot)

- **Automatic:** Mean selling price of **9.42 Lakhs INR**.

- **Manual:** Mean selling price of **3.93 Lakhs INR**.
- **\*Insight\*:** Automatic cars maintain a significant resale premium over manual equivalents.

## 5. Categorical Encoding & Correlation Matrix

I applied One-Hot Encoding to categorical variables using `pd.get_dummies(drop_first=True)` and rendered a Pearson correlation heatmap:

### Pearson Correlation Matrix Summary with Target (`Selling_Price`):

- `Present_Price` (Showroom Price): **+\$0.878\$** (Extremely strong positive correlation)
- `Fuel_Type_Diesel`: **+\$0.552\$** (Strong positive correlation)
- `Transmission_Automatic`: **+\$0.367\$** (Moderate positive correlation)
- `Seller_Type_Individual`: **-\$0.550\$** (Strong negative correlation; individual sellers list cheaper economy cars)
- `Car_Age`: **-\$0.236\$** (Negative correlation with age)
- `Kms_Driven`: **+\$0.029\$** (Weak direct correlation)

## 6. Scikit-Learn Pipeline & Train-Test Split

### 6.1 Stratified Split Strategy

- **Predictor Features (\$X\$):** `Present_Price`, `Kms_Driven`, `Car_Age`, `Fuel_Type`, `Seller_Type`, `Transmission`, `Owner`.
- **Target Variable (\$y\$):** `Selling_Price`.
- **Train-Test Ratio:** 80% Training (239 samples), 20% Testing (60 samples) with `random_state=42`.

### 6.2 Scikit-Learn `ColumnTransformer` Pipeline

To prevent data leakage during cross-validation, I wrapped numeric scaling and categorical encoding into a single reproducible `ColumnTransformer` pipeline:

```
num_cols = ['Present_Price', 'Kms_Driven', 'Car_Age', 'Owner']
cat_cols = ['Fuel_Type', 'Seller_Type', 'Transmission']

preprocessor = ColumnTransformer(
 transformers=[
 ('num', StandardScaler(), num_cols),
 ('cat', OneHotEncoder(drop='first', handle_unknown='ignore'), cat_cols)
]
)
```

## 7. Regression Model Comparison & Metrics

I trained and evaluated 5 diverse regression models under identical 80/20 train-test conditions:

- 14. **Linear Regression** (Ordinary Least Squares Baseline)
- 15. **Ridge Regression (\$lpha=1.0\$)** (\$L\_2\$ Regularized Linear Model)
- 16. **Decision Tree Regressor (`max\_depth=5`)**
- 17. **Random Forest Regressor (`n\_estimators=100`)** (Ensemble Bagging)
- 18. **Gradient Boosting Regressor (`n\_estimators=100`)** (Ensemble Boosting)

Comprehensive Regression Evaluation Table

Model Algorithm	MAE (Lakhs INR)	RMSE (Lakhs INR)	\$R^2\$ Score (Test)	5-Fold CV \$R^2\$ Mean	5-Fold CV \$R^2\$ Std
Random Forest Regressor	0.6210	1.3340	0.9634	0.9125	±0.0418
Gradient Boosting Regressor	0.6582	1.4120	0.9590	0.9084	±0.0452
Decision Tree Regressor	0.8415	1.8150	0.9321	0.8652	±0.0610
Ridge Regression	1.2180	2.1450	0.9052	0.8412	±0.0521
Linear Regression	1.2210	2.1520	0.9046	0.8398	±0.0534

## 8. Feature Importance & Model Diagnostics

### 8.1 Declared Best Model: Random Forest Regressor

- **Test \$R^2\$ Score: 0.9634** (Explains 96.34% of price variance on unseen test cars).
- **Mean Absolute Error (MAE): 0.6210 Lakhs** (~₹62,100 average prediction error).

### 8.2 Relative Feature Importances

Extracting feature importances from the Random Forest model revealed the key market price determinants:

Feature Name	Relative Importance Score	Market Interpretation
Present_Price	78.4%	Primary baseline price ceiling for used vehicles.
Car_Age	11.2%	Second strongest factor due to multi-year vehicle depreciation.
Kms_Driven	5.1%	Measures mechanical wear and tear.
Fuel_Type (Diesel)	2.8%	Fuel efficiency and engine torque premium.
Transmission (Automatic)	1.5%	Convenience feature price markup.
Seller_Type (Individual)	0.7%	Minor direct impact on listing price.
Owner	0.3%	Minimal impact compared to vehicle age and condition.

## 9. Model Serialization & Live Real-Time Predictions

I serialized the trained Random Forest model and `ColumnTransformer` preprocessor pipeline using `joblib`:

```
joblib.dump(best_model, 'best_car_price_model.pkl')
joblib.dump(preprocessor, 'car_preprocessor.pkl')
```

### Live Inference Test Results:

Vehicle Configuration	Present Price	Kms Driven	Car Age	Fuel Type	Seller	Transmission	Model Estimated Resale Price	Ground Truth Resale Price
Sample 1 (Maruti Ciaz)	9.54 Lakhs	43,000 km	7 yrs	Diesel	Dealer	Manual	7.12 Lakhs INR	7.25 Lakhs INR

Sample 2 (Hyundai i20)	4.15 Lakhs	15,000 km	4 yrs	Petrol	Individual	Manual	2.88 Lakhs INR	2.85 Lakhs INR
Sample 3 (Fortuner SUV)	30.61 Lakhs	50,000 km	5 yrs	Diesel	Dealer	Automatic	22.85 Lakhs INR	23.00 Lakhs INR

## 10. Personal Reflections & Internship Key Learnings

19. **Building Scikit-Learn Pipelines:** I learned how combining `StandardScaler` and `OneHotEncoder` inside a `ColumnTransformer` guarantees reproducible data transformations for deployment.
20. **Handling Non-Linear Depreciation:** Tree-based ensembles (Random Forest & Gradient Boosting) significantly outperformed linear regression models ( $0.963$  vs  $0.904$   $R^2$ ) by capturing steep initial depreciation curves.
21. **Domain Feature Engineering:** Deriving `Car_Age` from manufacturing year simplified model convergence and increased feature importance interpretability.